

K230 OpenCV (cv2) Introduction

This tutorial is based on CanMV_K230_YAHBOOM_micropython_v1.8-13 firmware. OpenCV examples require firmware version micropython_v1.8 or higher.

1. Why Use the OpenCV Library

K230 has two vision processing solutions: `cv_lite` and `cv2` (OpenCV). Here is their relationship and selection logic:

Comparison

	<code>cv_lite</code>	<code>cv2</code> (OpenCV)
Invocation method	Single function completes entire algorithm chain	Multi-step free combination
Typical code	<code>cv_lite.rgb888_find_blobs(img, thr, area, ksize)</code>	<code>cvtColor</code> → <code>GaussianBlur</code> → <code>Canny</code> → <code>findContours</code>
Flexibility	Low (each function is a black box)	High (each step is controllable and replaceable)
Debuggability	Difficult (intermediate results invisible)	Easy (every step can be displayed for inspection)
Feature breadth	~15 atomic operations	50+ standard OpenCV functions
Learning value	Hardware vendor lock-in	Universal skills, consistent with desktop OpenCV
Use case	Quick hardware verification, single-task production lines	Algorithm development, learning, prototype validation

Three Reasons to Choose cv2

1. Standard API, learn once use everywhere

K230's `cv2` is a MicroPython port of standard OpenCV - function names, parameters, and constants are consistent with the desktop version. Once you learn `cv2.Canny()` on K230, you can still write it on PC/Raspberry Pi/Jetson.

2. Free combination, no encapsulation limitations

`cv_lite`'s `find_blobs` internally does binarization + connected components + area filtering - if a certain step's parameters are not suitable, you cannot replace it or see the intermediate results. `cv2` lets you build your own pipeline: change the thresholding method, change the blur type, add morphology, fully customizable.

3. Broad coverage, one library for everything

50+ functions cover color conversion, geometric transformation, thresholding, filtering, edge detection, morphology, contours, Hough detection, template matching, arithmetic operations, channel operations, drawing - basically covering all embedded vision needs.

When to Choose cv_lite

If you only need a fixed task like "find a red color block and calculate distance", cv_lite is faster and more memory-efficient with one line of code. It compiles the entire pipeline into a C extension, avoiding multiple Python layer call overheads.

2. What is K230 OpenCV

OpenCV on K230 is a MicroPython port of standard OpenCV (`import cv2`), running on CanMV firmware. It provides OpenCV core image processing functions, captures real-time video through the camera (Sensor), and virtually displays in the IDE (Display).

3. Environment Requirements

Component	Description
Hardware	YAHBOOM K230 development board + camera module
Firmware	CanMV_K230_YAHBOOM_micropython_v1.8-13
IDE	CanMV IDE or VS Code + CanMV plugin
Resolution	Default 640x480

Reference: [K230 MicroPython Daily Build Download Center](#)

选择开发板

- CanMV K230 V1.0/V1.1
- CanMV K230 V3.0
- 庐山派 K230
- 庐山派 Lite K230D
- Yahboom K230**
- O1Studio K230
- O1Studio K230 Emmc
- Hiwonder K230
- WonderMK K230
- GT6700 K230
- BPI-K230D-Zero
- Junroc AI Cam



Yahboom K230

MicroPython Build for Yahboom K230

当前下载镜像:

CanMV_K230_YAHBOOM_micropython_v1.8-20-g9b151af_nncase_v2.11.0.img.gz

MD5: 66f0622fc6b8d22e5f552a40458d2fa

最新每日构建版本
Daily-0728
编译日期: 2026-07-28

[立即下载](#)

极简 USB 烧录指南 (推荐)

⚠ 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

- 准备工具**
下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。
- 进入烧录模式 (根据硬件情况选择)**

🔵 有 BOOT 键的板子

 - 按住 **BOOT** 键不放
 - 接入 USB 电源 (或按一下 **RESET** 键)
 - 软件识别到设备后即可松开

🔴 无 BOOT 键的板子

 - 先拔出 **SD** 卡
 - 接入 USB 电源, 使板子处于未启动状态
 - 重新插入 **SD** 卡, 软件将识别并进入烧录模式

4. Basic Program Structure

```
import time, os, sys, gc
from media.sensor import *      # Camera
from media.display import *    # Display
from media.media import *      # Media management
import cv2                     # OpenCV
import ulab.numpy as np        # Lightweight NumPy

# Initialize camera
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=640, height=480)
sensor.set_pixformat(Sensor.RGB888)

# Initialize display
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)                # Wait for sensor to stabilize

# Main loop
try:
    while True:
        time.sleep_ms(10)      # CPU yield (IDE can stop)
        img = sensor.snapshot()
        img_np = img.to_numpy_ref()

        # cv2 image processing ...
        gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
```

```

edges = cv2.Canny(gray, 80, 200)

# Grayscale display
img_out = image.Image(640, 480, image.GRAYSCALE,
                      alloc=image.ALLOC_REF, data=edges)
Display.show_image(img_out)
gc.collect()
finally:
    sensor.stop()
    Display.deinit()

```

5. Display Methods

Output type	Display method
Color image	<code>cv2.xxx(img_np, ...)</code> → <code>Display.show_image(img)</code>
Grayscale image	<code>image.Image(w,h,image.GRAYSCALE,alloc=image.ALLOC_REF,data=out)</code> → <code>Display.show_image(img_out)</code>
⚠ Color image + <code>image.Image</code>	K230 does not support <code>image.Image(GB888, ALLOC_REF, data=cv2_output)</code>

6. K230 Specific Limitations

Limitation	Description
<code>cv2.add</code> does not support scalar broadcasting	Both arrays must have the same size
<code>cv2.compare</code> requires float64	Scalars must be floating point
<code>HoughCircles</code> returns (N,3)	Not standard OpenCV's (N,1,3)
<code>HoughLines</code> returns (N,2)	Not (N,1,2)
<code>approxPolyDP</code> returns (N,2)	Not (N,1,2)
<code>image.Image(GB888, ALLOC_REF)</code>	Can only use sensor's native img object
<code>np.float32</code> does not exist	ulab only supports <code>np.float</code>
<code>np.array()</code> dtype keyword	May error in loops, recommend omitting
<code>cv2.putText</code> grayscale scalar color	Use (255,) tuple instead of 255

7. Performance Reference

Operation type	640x480 frame rate	Optimization methods
Pixel operations (flip/resize/threshold)	50-80 fps	—

Operation type	640x480 frame rate	Optimization methods
Local convolution (blur/morphology/edge)	15-30 fps	Reduce kernel size
Global search (Hough/matchTemplate)	3-10 fps	Downsample to 320x240 for processing

8. opencv_demo Collection

17 independently runnable camera demos, each corresponding to an original OpenCV file, covering 50+ cv2 functions.

Learning Path

01 Basic Transforms → 08 Drawing → 12 Thresholding

|

└→ 10 Blur → 11 Edge Detection

|

└→ 02 Morphology

|

└→ 13 Color Segmentation → 03 Shape Classification → 07 Contour Analysis → 04

Corners

|

└→ 05 Hough Circles → 06 Hough Lines → 09 Template Matching

|

└→ 14 Pixel Transform

└→ 15 Arithmetic Bitwise Operations

└→ 16 Channel Statistics